

Consignas del día 2

Cada consigna está escrita para leerse sola, sin depender de haber escuchado la explicación.

Consigna 4 — Ejecución cruzada

Cuándo: día 2, 9:30. **Modalidad:** individual, recorriendo el aula.

1. Cada pareja deja su experimento abierto en el Builder, en su computadora, listo para correr.
2. Tu pareja corre los experimentos de la pareja anterior y de la siguiente, en ronda (en el pizarrón: B corre A y C; A corre la última y B). Andá a sus computadoras.
3. Corré con **tu** código de participante (007 , por ejemplo). Hacé la tarea en serio: son los datos de otra persona.
4. No hace falta subir nada: el CSV queda en la carpeta `data/` de lxs autorxs.
5. Si terminás antes, seguí con los experimentos de otras parejas hasta que se termine el tiempo: cada uno suma participantes.

Después del almuerzo cada pareja va a abrir los CSV que le generaron.

Consigna 5 — Una marca en el momento justo

Cuándo: día 2, 13:30, después del almuerzo: primero la explicación y después la práctica. **Modalidad:** de a dos, sobre tu experimento. **Referencia:** `dia2/marcas/` y el demo de la mañana.

Objetivo

Que tu experimento mande una marca **en el instante en que el estímulo aparece en pantalla**, con un código por condición, y verificarlo. Sin hardware: la marca va al `.log`.

Parte A — Un código por condición (10 min)

Agregá a tu `condiciones.csv` una columna `trigger_code`: un número distinto por condición (por ejemplo 11, 12, 21, 22: primer dígito la congruencia o la dificultad, segundo el lado o el color).

Pregunta: ¿por qué conviene un código por condición en vez de mandar siempre `1`?

Parte B — La marca (25 min)

En la rutina `trial`, agregá un componente **Code debajo** del estímulo:

```
# Begin Experiment
def mandar_marca(codigo):
    logging.data(f'TRIGGER {codigo}')

# Begin Routine
marca_pendiente = True

# Each Frame (cambiá `palabra` por el nombre de tu estímulo)
if palabra.status == STARTED and marca_pendiente:
    win.callOnFlip(mandar_marca, int(trigger_code))
    marca_pendiente = False

# End Routine
thisExp.addData('trigger_code', int(trigger_code))
```

`win.callOnFlip` no manda la marca ahora: la agenda para **justo después del próximo flip**, que es cuando el estímulo aparece de verdad. Es lo que hace por dentro el componente *Parallel Port Out* del Builder.

Parte C — Verificarlo (15 min)

Corré el experimento y abrí el `.log` que quedó en `data/`. Buscá una línea `TRIGGER` y, justo abajo, la línea donde el estímulo se prende (`autoDraw = True`): tienen la misma hora, o casi.

- ☐ Hay **una** línea `TRIGGER` por trial, ni cero ni dos
- ☐ El código cambia según la condición
- ☐ La marca está pegada al momento en que aparece el estímulo, no al final del trial
- ☐ En el CSV aparece la columna `trigger_code`

Preguntas

1. Si en vez de `win.callOnFlip(...)` llamás directamente a `mandar_marca(...)` en *Each Frame*, ¿la marca sale antes o después de que el estímulo aparezca? ¿Por cuánto, como máximo?
 2. El `.log` dice que la marca salió a los 0.502 s y que el estímulo apareció a los 0.500 s. ¿Eso prueba que están sincronizados? (*Pista: ¿quién escribió las dos líneas?*)
-

Consigna 6 — Analizar los datos de la clase

Cuándo: día 2, 15:00. **Modalidad:** de a dos. **Archivo de trabajo:** `dia2/analisis/ejercicio_dia2.py`

Preparación

1. Bajá del Classroom el archivo `datos_clase.zip` y descomprimilo dentro de `dia2/analisis/`, al lado del ejercicio: tienen que quedar los CSV del Stroop mínimo de toda la clase en `dia2/analisis/datos_clase/`. Es lo único que se baja aparte: el resto ya estaba en el ZIP del primer día.
2. Abrí `dia2/analisis/ejercicio_dia2.py` en el Coder. Hay cinco bloques con `TOD0`: se completan en orden.

Parte 1 — Mirar antes de calcular

- a. ¿Cuántas filas y columnas tiene la tabla completa?
- b. La lista de nombres de columna.
- c. Las primeras y las últimas 5 filas.
- d. Clasificá cada columna en uno de tres grupos: viene del archivo de condiciones / la registró el trial / es información de la sesión.

Control: el número de filas tiene que ser `n_participantes × 36`. Si no da, averiguá por qué **antes** de seguir.

Comparación: abrí también uno de los CSV que te dejaron en tu experimento de Builder. ¿Tiene filas que no son trials? ¿Por qué este no?

Parte 2 — Describir

- a. ¿Cuántos trials tiene cada participante?
- b. Por participante: RT medio, RT mediano y porcentaje de aciertos.
- c. **¿La media y la mediana dan lo mismo? ¿Por qué no?**

Parte 3 — Graficar

Sobre los datos ya limpios:

- a. Dos histogramas de `response_time` superpuestos, uno por valor de `congruent`.
- b. Una línea vertical en la mediana de cada uno.
- c. Guardá la figura como PNG.

Si matplotlib se hace cuesta arriba, salteá a la parte 4 y volvé después. Graficar no es el objetivo del curso.

Parte 4 — ¿Hay efecto Stroop?

- a. El efecto Stroop medio del grupo, **en milisegundos**.
- b. ¿En cuántxs participantes el efecto es positivo?
- c. Un t-test pareado entre `rt_incongruent` y `rt_congruent` (`scipy.stats.ttest_rel`). Imprimí `t` y `p`.
- d. La conclusión en **una** frase, con el `n`.

Parte 5 — Guardar

Guardá la tabla de efectos por participante como CSV, y explicá en un comentario por qué conviene guardarla además de los datos crudos.

Preguntas de cierre

1. La limpieza descarta los trials con RT menor a 0.2 s o mayor a 2.0 s. ¿De dónde salen esos números? Probá con 0.15 y 2.5 y mirá cuánto se mueve el resultado.
2. ¿Por qué se promedia primero dentro de cada participante y recién después entre participantes?
3. ¿Tu efecto se parece a los 70–100 ms de la literatura? Si no, ¿qué explicaciones se te ocurren?

Si terminás antes

1. El efecto de cada participante como líneas individuales.
2. ¿Los aciertos también muestran efecto Stroop, o solo el RT?
3. Lo mismo con los datos del experimento propio.

Solución: se proyecta a las 16:30. Intentalo antes: equivocarse acá es la parte que más enseña.